



Experiment 09

Stored Procedures in PostgreSQL

Name: Deven

UID: 24BCF10060

Aim

To study and implement PostgreSQL **stored procedures** using IN/OUT parameters, conditional (IF...ELSE) logic, RAISE NOTICE messages, and multi-table INSERT/UPDATE operations to automate real-world business transactions.

Question 1 – Gender-wise Employee Count (Medium)

An organization stores employee details in the `employee_info` table. Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter and calculates the total number of employees belonging to that gender. The calculated count should be returned through an OUT parameter and should also be displayed as a message using RAISE NOTICE. For example, if Male is passed to the procedure, it should count all employees whose gender is Male.

Given Data

```
1 CREATE TABLE employee_info
2 (
3     id        SERIAL PRIMARY KEY,
4     name      VARCHAR(50),
5     gender    VARCHAR(10),
6     salary    NUMERIC(10,2),
7     city      VARCHAR(50)
8 );
9
10 INSERT INTO employee_info (name, gender, salary, city) VALUES
11 ('Aarav Sharma', 'Male', 45000, 'Delhi'),
12 ('Diya Patel', 'Female', 52000, 'Mumbai'),
13 ('Rohan Mehta', 'Male', 38000, 'Pune'),
14 ('Sneha Reddy', 'Female', 60000, 'Hyderabad'),
15 ('Karan Malhotra', 'Male', 47000, 'Chandigarh'),
16 ('Isha Kapoor', 'Female', 55000, 'Delhi'),
17 ('Vivaan Joshi', 'Male', 42000, 'Bangalore');
```

Answer

```
1 CREATE OR REPLACE PROCEDURE pr_count_by_gender(
2     IN  p_gender VARCHAR,
3     OUT p_count  INT
4 )
5 LANGUAGE plpgsql
6 AS $$
7 BEGIN
8     SELECT COUNT(*)
9     INTO p_count
10    FROM employee_info
11    WHERE gender = p_gender;
12
```



```
13 RAISE NOTICE 'Total % employees: %', p_gender, p_count;  
14 END;  
15 $$;
```

Calling the procedure

```
1 CALL pr_count_by_gender('Male', NULL);  
2 CALL pr_count_by_gender('Female', NULL);
```

Output

```
NOTICE: Total Male employees: 4  
NOTICE: Total Female employees: 3
```

p_count
4

Result of CALL pr_count_by_gender('Male', NULL); – the OUT parameter p_count is returned as a single-row result.



Question 2 – Purchase Processing Procedure (Hard)

An e-commerce company maintains product stock information in the `products` table and stores completed orders in the `sales` table. Create a PostgreSQL stored procedure named `pr_buy_products` that accepts a product name and the required quantity as input parameters and processes the purchase only when sufficient stock is available.

The procedure should:

- Check whether the requested product exists and has sufficient stock.
- If sufficient stock is available:
 - Retrieve its product code and price.
 - Calculate the total sale amount as $\text{Sale Price} = \text{Product Price} \times \text{Quantity Ordered}$.
 - Insert a new record into the `sales` table.
 - Reduce `quantity_remaining` by the purchased quantity.
 - Increase `quantity_sold` by the purchased quantity.
 - Display a success message.
- If sufficient stock is *not* available:
 - Do not insert or update any record.
 - Display an insufficient-quantity message.

Given Data

```
1 CREATE TABLE products
2 (
3     product_code      VARCHAR(10) PRIMARY KEY,
4     product_name      VARCHAR(100),
5     price              NUMERIC(10,2),
6     quantity_remaining INT,
7     quantity_sold     INT
8 );
9
10 CREATE TABLE sales
11 (
12     order_id          SERIAL PRIMARY KEY,
13     order_date         DATE,
14     product_code      VARCHAR(10),
15     quantity_ordered  INT,
16     sale_price         NUMERIC(10,2),
17     FOREIGN KEY(product_code) REFERENCES products(product_code)
18 );
19
20 INSERT INTO products VALUES
21 ('P001','Wireless Mouse',699.00,50,0),
22 ('P002','Mechanical Keyboard',2499.00,20,0),
23 ('P003','USB-C Hub',1299.00,5,0),
24 ('P004','Webcam 1080p',1899.00,15,0);
```



Answer

```
1 CREATE OR REPLACE PROCEDURE pr_buy_products(  
2     IN p_product_name VARCHAR,  
3     IN p_quantity      INT  
4 )  
5 LANGUAGE plpgsql  
6 AS $$  
7 DECLARE  
8     v_product_code VARCHAR(10);  
9     v_price          NUMERIC(10,2);  
10    v_available      INT;  
11    v_total          NUMERIC(10,2);  
12 BEGIN  
13     SELECT product_code, price, quantity_remaining  
14     INTO v_product_code, v_price, v_available  
15     FROM products  
16     WHERE product_name = p_product_name;  
17  
18     IF NOT FOUND THEN  
19         RAISE NOTICE 'Product "%" not found.', p_product_name;  
20         RETURN;  
21     END IF;  
22  
23     IF v_available >= p_quantity THEN  
24  
25         v_total := v_price * p_quantity;  
26  
27         INSERT INTO sales (order_date, product_code, quantity_ordered  
28         , sale_price)  
29         VALUES (CURRENT_DATE, v_product_code, p_quantity, v_total);  
30  
31         UPDATE products  
32         SET quantity_remaining = quantity_remaining - p_quantity,  
33           quantity_sold       = quantity_sold + p_quantity  
34         WHERE product_code = v_product_code;  
35  
36         RAISE NOTICE 'Order placed: % x "%" for a total of %.',  
37         p_quantity, p_product_name, v_total;  
38     ELSE  
39         RAISE NOTICE 'Insufficient stock for "%": requested %, only %  
40         available.',  
41         p_product_name, p_quantity, v_available;  
42     END IF;  
43 END;  
44 $$;
```

Demonstration calls

```
1 CALL pr_buy_products('Wireless Mouse', 5);  
2 CALL pr_buy_products('USB-C Hub', 3);  
3 CALL pr_buy_products('USB-C Hub', 10);
```

Output

```
NOTICE: Order placed: 5 x "Wireless Mouse" for a total of 3495.00.
NOTICE: Order placed: 3 x "USB-C Hub" for a total of 3897.00.
NOTICE: Insufficient stock for "USB-C Hub": requested 10, only 2 available.
```

The first two calls succeed (5 units of Wireless Mouse and 3 units of USB-C Hub, both within available stock) and each inserts a row into **sales** while updating **products**. The third call requests 10 units of USB-C Hub when only 2 remain after the second purchase, so the procedure leaves both tables unchanged and only raises a notice.

products table after the calls above

product_code	product_name	price	qty_remaining	qty_sold
P001	Wireless Mouse	699.00	45	5
P002	Mechanical Keyboard	2499.00	20	0
P003	USB-C Hub	1299.00	2	3
P004	Webcam 1080p	1899.00	15	0

sales table after the calls above

order_id	order_date	product_code	qty_ordered	sale_price
1	2026-09-14	P001	5	3495.00
2	2026-09-14	P003	3	3897.00

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood the difference between a PostgreSQL stored **procedure** (invoked with **CALL**, can modify data and commit) and a function.
- Learned to declare and use **IN** and **OUT** parameters to pass values into a procedure and return a computed value from it.
- Used **RAISE NOTICE** to display formatted, human-readable status messages from within PL/pgSQL.
- Applied **DECLARE** and **SELECT ... INTO** to capture intermediate values (product code, price, available stock) inside a procedure body.
- Implemented conditional business logic with **IF ... THEN ... ELSE ... END IF** to allow or block a transaction based on stock availability.
- Performed a coordinated **INSERT** (new sale) and **UPDATE** (stock adjustment) inside a single procedure call, keeping the two tables consistent.
- Gained practical exposure to automating real-world transactional workflows (inventory and order processing) using PostgreSQL stored procedures.